

교과목 4 : AI 활용 애플리케이션 개발

단원 2 : Django Framework

DAY1. Django 기반 웹 서버 개발 기초

목 차

1. Django란?		3
2. Django의 주요 특징		5
3. MVT 디자인 패턴		7
4. Django 프로젝트 기본 구조		13

1. Django란?

Django 는 **Python** 으로 **웹 애플리케이션을 개발하기 위한 대표적인 서버 측(Web Backend) 프레임워크**입니다. 회원 관리, 게시판, 관리자 페이지, 데이터베이스 연동, REST API 같은 웹 서비스 기능을 체계적으로 개발할 수 있도록 다양한 기능을 기본 제공합니다.

특히 Django 는 “**빠른 개발(Rapid Development)**”과 “**재사용 가능한 구조**”를 중요하게 설계되었으며, 비교적 규모가 큰 웹 서비스에도 사용할 수 있습니다.

1. Django 란?

Django 는 Python 기반의 **오픈소스 웹 프레임워크(Web Framework)**입니다.

웹 애플리케이션을 직접 개발하려면 일반적으로 다음 기능들을 구현해야 합니다.

- URL 요청 처리
- HTML 화면 생성
- 데이터베이스 연결
- 회원 인증
- 세션 관리
- 관리자 기능
- 보안 처리
- 폼 데이터 검증

Django 는 이러한 기능의 상당 부분을 프레임워크 차원에서 제공합니다.

예를 들어 사용자가 다음 주소로 접속했다고 가정하겠습니다.

http://localhost:8000/board/

Django 에서는 대략 다음 흐름으로 요청을 처리합니다.

사용자

↓

웹 브라우저

↓ HTTP Request

Django

↓

URL Routing

↓

View

↓

Model ↔ Database

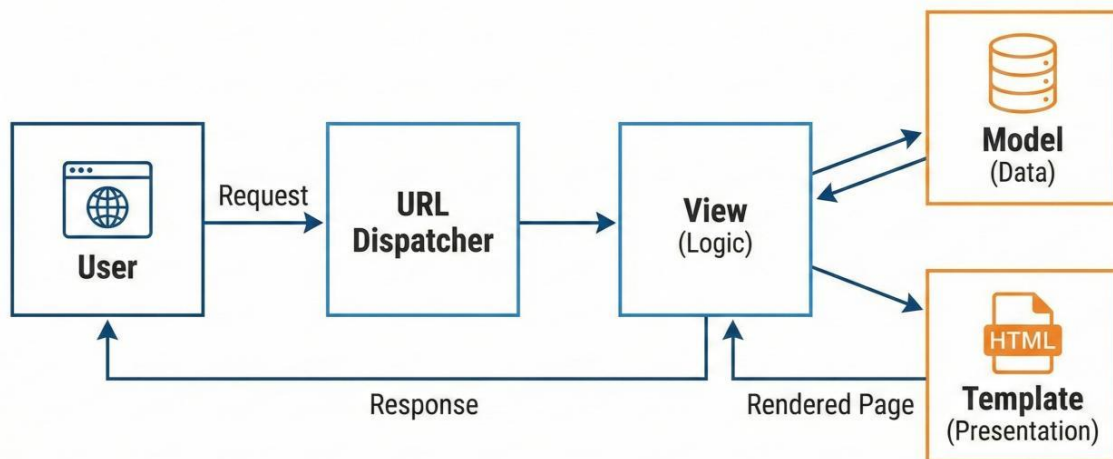
↓

Template

↓ HTTP Response

웹 브라우저

Django MVT Architecture & Data Flow



2. Django의 주요 특징

1 Python 기반

Django 는 Python 으로 개발합니다.

```
def hello(request):  
    return HttpResponse("Hello Django")
```

Python 의 비교적 간결한 문법을 그대로 활용할 수 있다는 장점이 있습니다.

2 MVT 구조

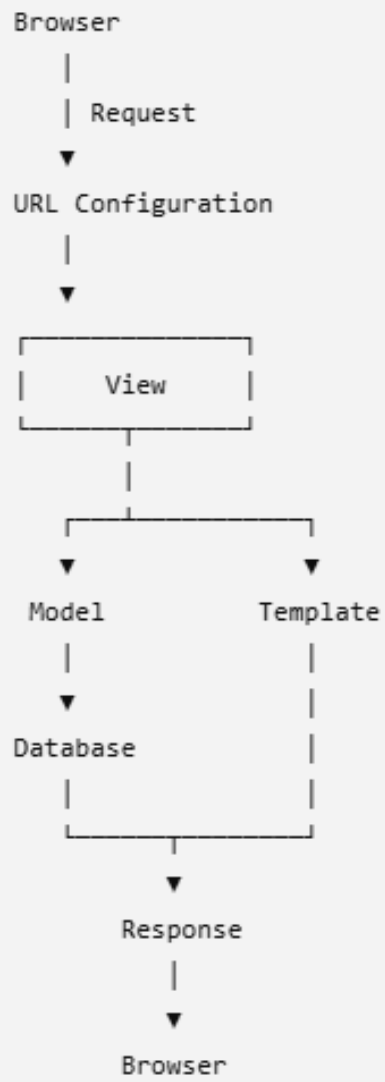
Django 의 가장 중요한 개념 중 하나가 **MVT(Model-View-Template)** 구조입니다.

일반적인 웹 프레임워크의 MVC(Model-View-Controller) 패턴과 비슷하지만 Django 에서는 이를 MVT 라고 설명합니다.

각 구성 요소의 역할은 다음과 같습니다.

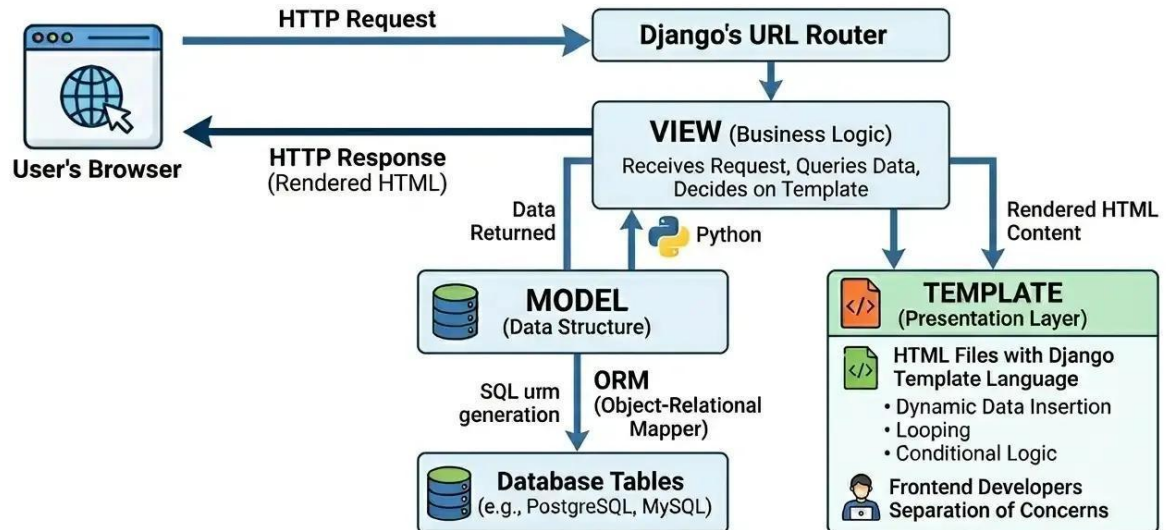
구성 요소	역할
Model	데이터와 데이터베이스 처리
View	요청 처리 및 비즈니스 로직
Template	HTML 화면 표현
URLconf	URL 과 View 연결

Django MVT



3. MVT 디자인 패턴

DJANGO'S REQUEST FLOW: THE MVT ARCHITECTURE



1. Model

Model 은 데이터베이스의 데이터를 Python 객체 형태로 정의하고 관리하는 부분입니다.

예를 들어 게시판을 만든다면 게시글을 다음과 같이 정의할 수 있습니다.

```
from django.db import models

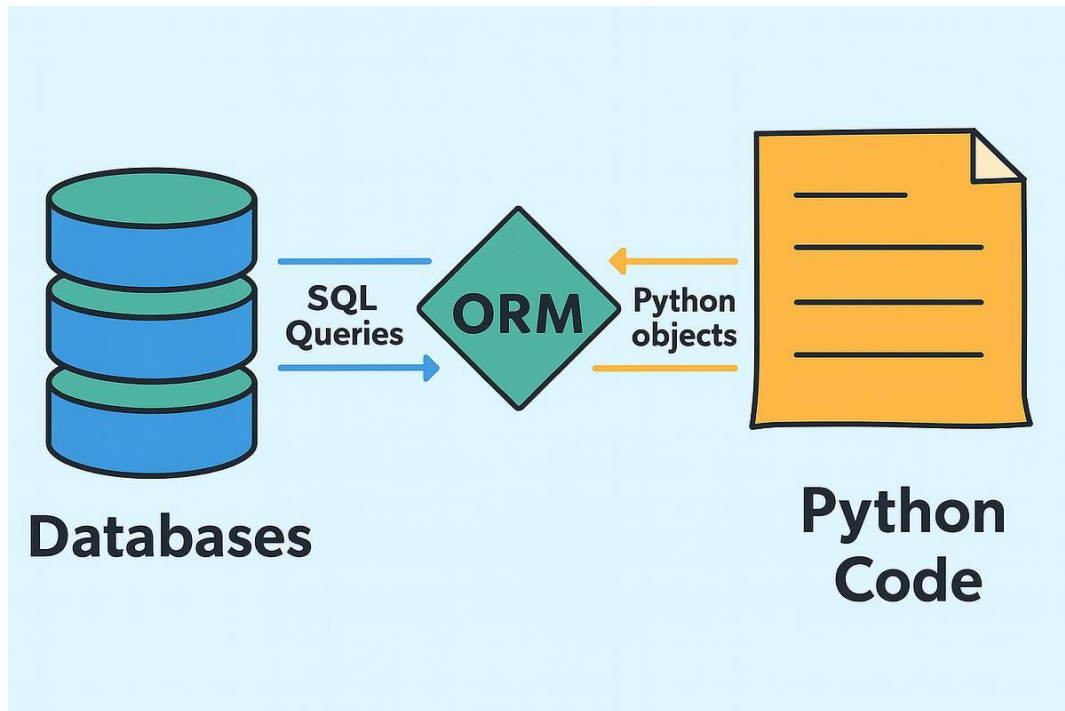
class Board(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    writer = models.CharField(max_length=50)
    created_at = models.DateTimeField(auto_now_add=True)
```

일반적인 관계형 데이터베이스 관점에서 보면 다음과 대응됩니다.

Python Class	Database
Board	→ BOARD 테이블

title	→	TITLE 컬럼
content	→	CONTENT 컬럼
writer	→	WRITER 컬럼
created_at	→	CREATED_AT 컬럼

즉, Python 클래스를 이용하여 데이터베이스 테이블 구조를 표현할 수 있습니다.



2. Django ORM

Django 의 중요한 기능 중 하나가 **ORM(Object Relational Mapping)**입니다.

ORM 을 사용하면 SQL 을 직접 작성하지 않고 Python 객체를 이용하여 데이터베이스를 처리할 수 있습니다.

예를 들어 SQL 에서는 다음과 같이 작성합니다.

```
SELECT *
FROM board
WHERE id = 1;
```

Django ORM 에서는 다음과 같이 작성할 수 있습니다.

```
board = Board.objects.get(id=1)
```


전체 데이터를 가져올 수도 있습니다.

```
boards = Board.objects.all()
```

데이터를 저장할 수도 있습니다.

```
Board.objects.create(
    title="Django 공부",
    content="Django ORM 실습",
    writer="홍길동"
)
```

조건 검색도 가능합니다.

```
boards = Board.objects.filter(writer="홍길동")
```

따라서 Django에서는

```
Python 객체
  ↓
Django ORM
  ↓
SQL
  ↓
Database
```

와 같은 방식으로 데이터베이스를 사용할 수 있습니다.

3. View

View는 사용자의 HTTP 요청을 받아 **실제 처리 로직을 수행하는 부분**입니다.

예를 들어 다음과 같이 작성할 수 있습니다.

```
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello Django")
```

사용자가 /hello/를 요청하면 이 함수가 실행되도록 URL 을 연결할 수 있습니다.

좀 더 실제적인 예를 살펴보겠습니다.

```
from django.shortcuts import render
from .models import Board

def board_list(request):
    # 모든 게시글 조회
    boards = Board.objects.all()

    # HTML Template 으로 데이터 전달
    return render(
        request,
        "board/list.html",
        {"boards": boards}
    )
```

처리 과정은 다음과 같습니다.

```
사용자 요청
GET /board/
    ↓
board_list()
    ↓
Board.objects.all()
    ↓
Database 조회
    ↓
boards
    ↓
list.html
    ↓
HTML Response
```

4. URL Routing

Django 에서는 URL 과 View 를 연결해야 합니다.

urls.py 가 이 역할을 담당합니다.

```
from django.urls import path
from . import views

urlpatterns = [
    path(
        "hello/",
        views.hello
    ),
    path(
        "board/",
        views.board_list
    ),
]
```

예를 들어

```
http://localhost:8000/hello/
```

요청은

```
views.hello
```

함수와 연결됩니다.

그리고

```
http://localhost:8000/board/
```

요청은

```
views.board_list
```

함수와 연결됩니다.

5. Template

Template 은 사용자에게 보여줄 **HTML 화면을 담당**합니다.

Django Template Language 를 사용하면 Python 에서 전달받은 데이터를 HTML 에 출력할 수 있습니다.

```
<!DOCTYPE html>
<html>
<head>
    <title>게시판</title>
</head>
<body>
<h1>게시판</h1>
{% for board in boards %}
    <h3>{{ board.title }}</h3>
    <p>
        {{ board.content }}
    </p>
{% endfor %}
</body>
</html>
```

여기서

```
{{ board.title }}
```

은 데이터를 출력하는 표현식입니다.

그리고

```
{% for board in boards %}
```

는 Template 에서 사용하는 제어문입니다.

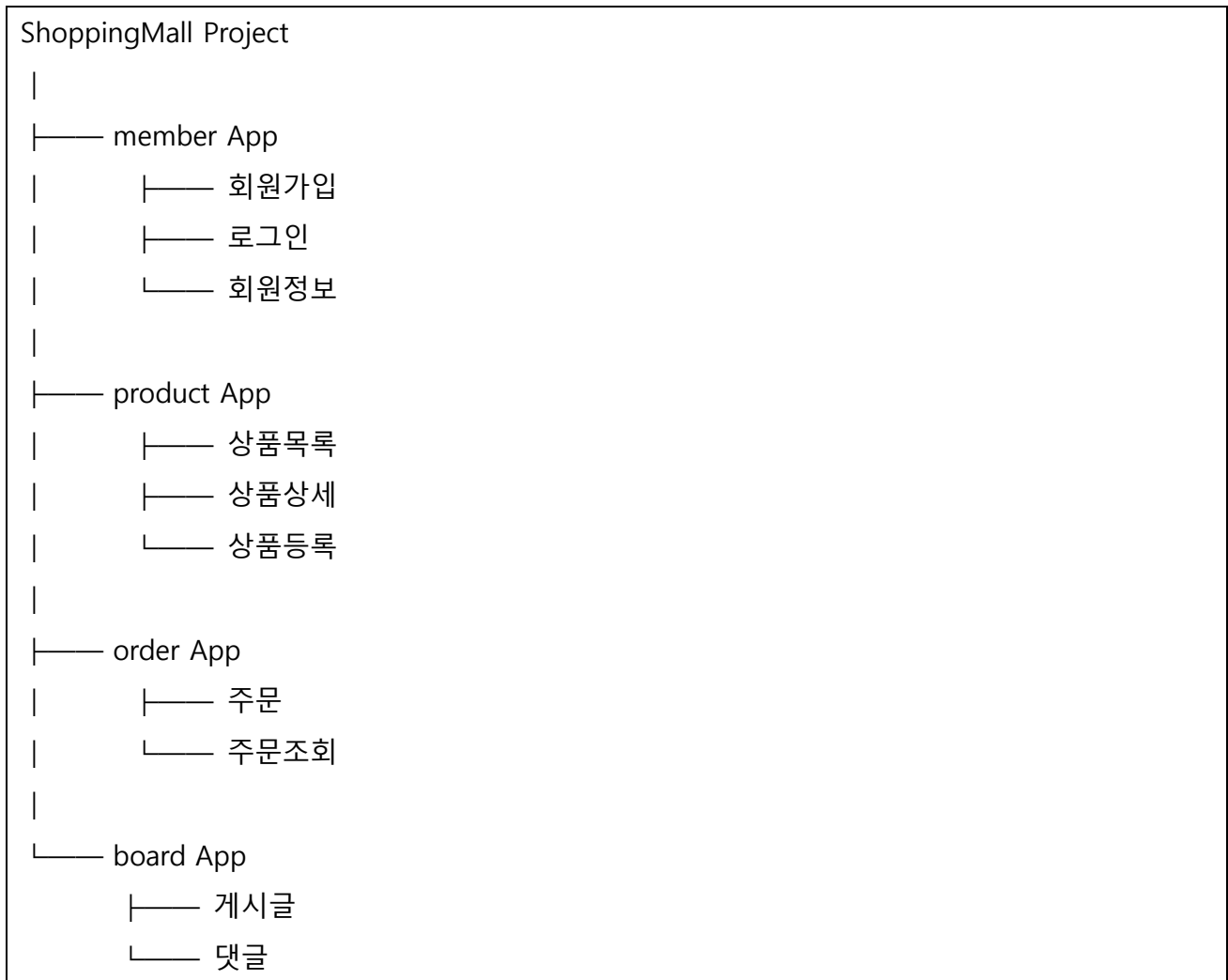
4. Django 프로젝트 기본 구조

1. Django 의 Project 와 App

Django 를 공부할 때 반드시 구분해야 하는 개념입니다.

Django 에서는 전체 웹 서비스를 **Project**, 세부 기능을 **App** 으로 구성합니다.

예를 들어 쇼핑몰을 개발한다면 다음과 같이 구성할 수 있습니다.

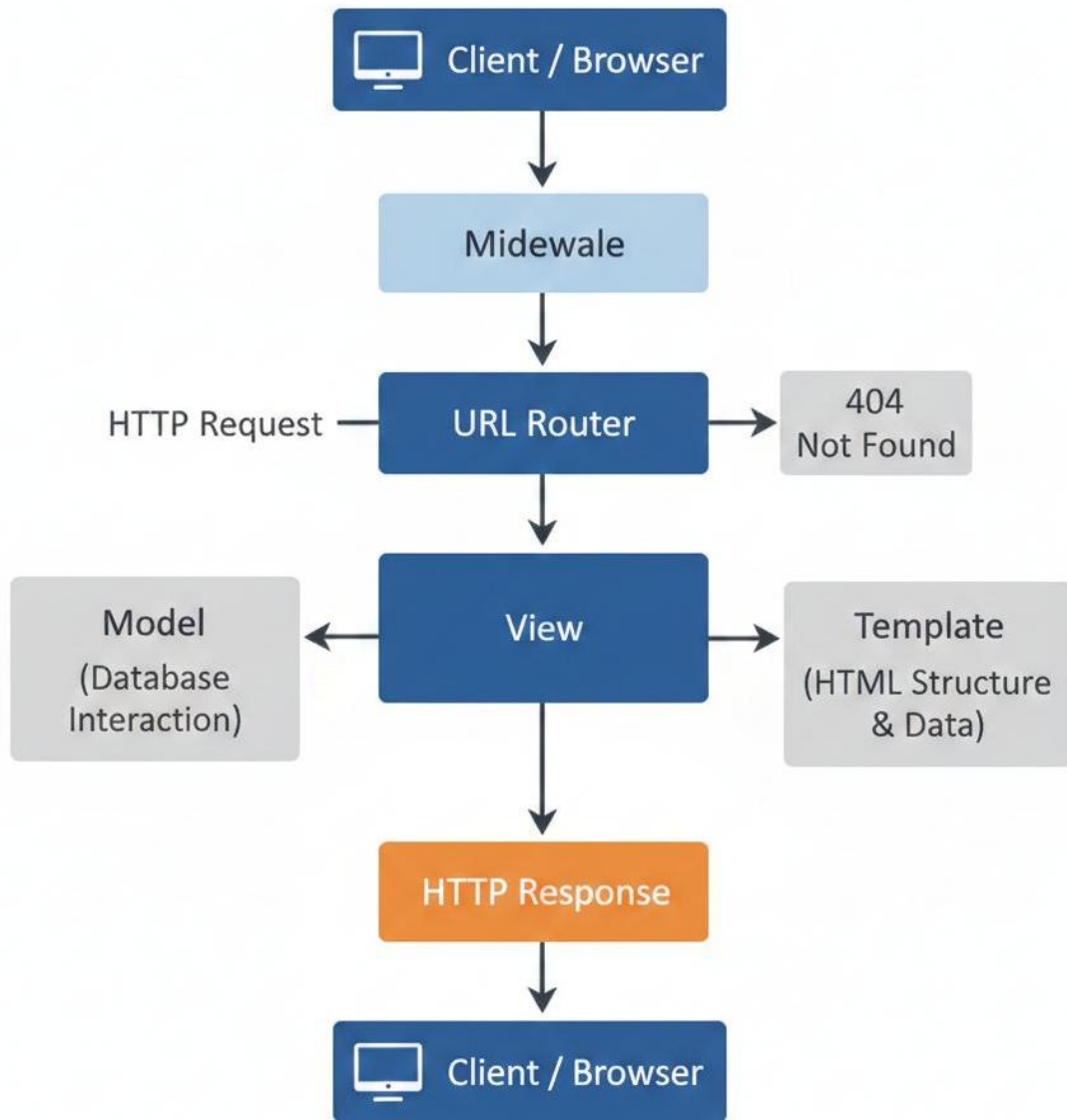


즉,

Project = 전체 웹 서비스

App = 특정 기능을 담당하는 모듈

이라고 이해하면 좋습니다.



2. Django 프로젝트 기본 구조

Django 프로젝트를 만들면 일반적으로 다음과 같은 구조를 사용합니다.

```
myproject/ |
|—— manage.py
|—— myproject/
|   |—— __init__.py
|   |—— settings.py
|   |—— urls.py
|   |—— asgi.py
```

```

|   └── wsgi.py
└── board/
    ├── migrations/
    ├── templates/
    |
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── urls.py
    └── views.py

```

주요 파일의 역할은 다음과 같습니다.

파일	역할
manage.py	Django 프로젝트 관리 명령 실행
settings.py	프로젝트 전체 환경설정
urls.py	URL 설정
models.py	데이터베이스 Model
views.py	요청 처리
admin.py	관리자 사이트 설정
apps.py	App 설정
tests.py	테스트 코드
migrations/	DB 변경 이력 관리
templates/	HTML Template

3. Django 프로젝트 생성

Django 를 설치합니다.

```
pip install django
```

설치 버전을 확인합니다.

```
django-admin --version
```

프로젝트를 생성합니다.

```
django-admin startproject myproject
```

프로젝트 디렉터리로 이동합니다.

```
cd myproject
```

서버를 실행합니다.

```
python manage.py runserver
```

기본적으로 다음 주소에서 개발 서버를 확인할 수 있습니다.

```
http://127.0.0.1:8000/
```

4. Django App 생성

게시판 App 을 만든다고 가정하겠습니다.

```
python manage.py startapp board
```

그러면 다음 구조가 만들어집니다.

```
board/  
|  
├── migrations/  
├── __init__.py  
├── admin.py  
├── apps.py  
├── models.py  
├── tests.py  
└── views.py
```

그리고 settings.py 에 App 을 등록합니다.

```
INSTALLED_APPS = [  
    # Django 기본 App  
    'django.contrib.admin',  
    'django.contrib.auth',
```



```
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',

# 개발자가 만든 App
'board',

]
```

5. Migration

Django 에서 Model 을 만들었다고 해서 데이터베이스 테이블이 바로 만들어지는 것은 아닙니다.

Model 변경 내용을 Migration 으로 생성해야 합니다.

```
python manage.py makemigrations
```

그다음 실제 데이터베이스에 적용합니다.

```
python manage.py migrate
```

전체 흐름은 다음과 같습니다.

```
models.py 작성
    ↓
makemigrations
    ↓
Migration 파일 생성
    ↓
migrate
    ↓
Database 반영
    ↓
Table 생성/변경
```

이 개념은 Django 데이터베이스 개발에서 매우 중요합니다.

6. Django Admin

Django 의 큰 장점 중 하나는 관리자 페이지를 기본 제공한다는 것입니다.

관리자 계정을 생성합니다.

```
python manage.py createsuperuser
```

서버를 실행합니다.

```
python manage.py runserver
```

관리자 페이지에 접속합니다.

```
http://127.0.0.1:8000/admin/
```

Model 을 관리자 페이지에서 사용하려면 admin.py 에 등록합니다.

```
from django.contrib import admin
from .models import Board

admin.site.register(Board)
```

이후 관리자 화면에서 게시글을 추가·수정·삭제할 수 있습니다.

7. Django 의 데이터 처리 흐름

게시판 조회를 예로 들어 전체 동작을 정리하면 다음과 같습니다.

- ① Browser
 - | GET /board/
 - ▼
- ② urls.py
 - | URL Matching
 - ▼
- ③ views.py
 - | Board.objects.all()
 - ▼
- ④ models.py
 - | ORM



따라서 Django 를 처음 학습할 때는 다음 관계를 정확히 이해하는 것이 중요합니다.



8. Django 의 주요 기능

Django 는 단순히 HTML 페이지를 출력하는 프레임워크가 아닙니다. 실제 웹 서비스 개발에 필요한 여러 기능을 제공합니다.

기능	설명
URL Routing	URL 과 View 연결
ORM	Python 객체 기반 DB 처리
Template	동적 HTML 생성
Forms	입력 Form 처리
Authentication	로그인/인증
Authorization	사용자 권한 관리
Session	사용자 상태 관리
Admin	관리자 사이트
Middleware	요청/응답 중간 처리
Migration	DB 구조 변경 관리
Security	주요 웹 보안 기능
Testing	웹 애플리케이션 테스트

9. Django 와 FastAPI 비교

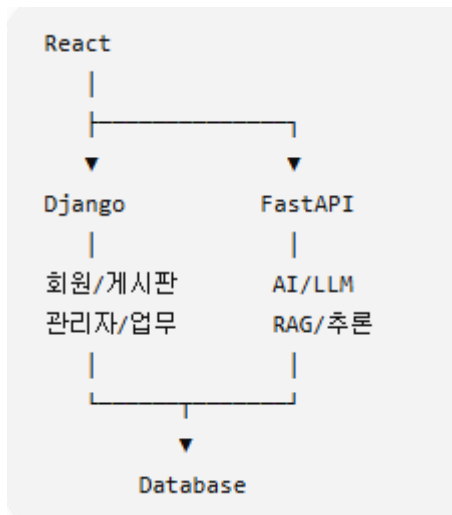
Python 웹 개발을 공부한다면 Django 와 FastAPI 의 차이도 이해할 필요가 있습니다.

구분	Django	FastAPI
주요 목적	종합 웹 애플리케이션	API 서버
구조	MVT 중심	API 중심
ORM	기본 제공	별도 선택 가능
Template	기본 제공	선택적
Admin	기본 제공	기본 제공하지 않음
인증	다양한 기능 제공	직접 구성하는 경우가 많음
API 문서	별도 구성	OpenAPI/Swagger 자동화 강점
비동기 처리	지원	핵심 설계에 적극 활용
대표 활용	쇼핑몰, 게시판, 업무시스템	REST API, AI/ML API, 마이크로서비스

따라서 웹 사이트의 백엔드 기능 전체를 하나의 프레임워크에서 체계적으로 구성하려면 Django 가 편리합니다.

반면 AI 모델 추론 API, LLM API, 마이크로서비스처럼 API 중심으로 개발하는 경우 FastAPI 가 특히 편리합니다.

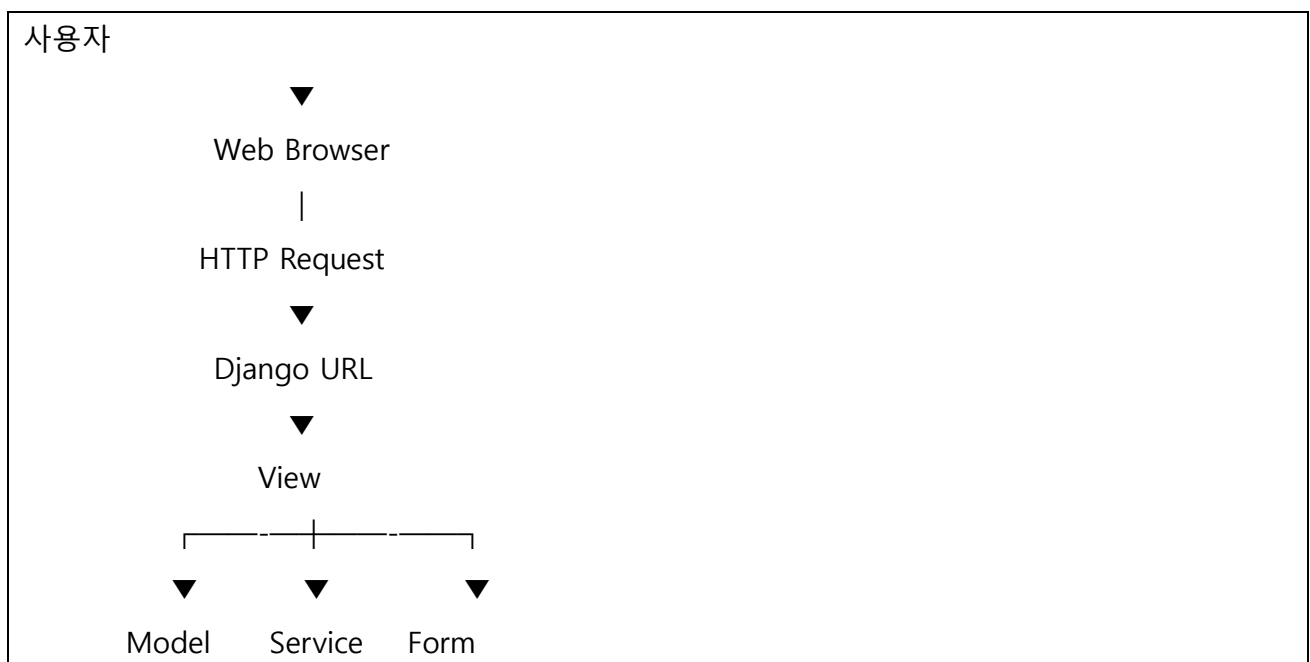
두 프레임워크를 반드시 경쟁 관계로 볼 필요도 없습니다.

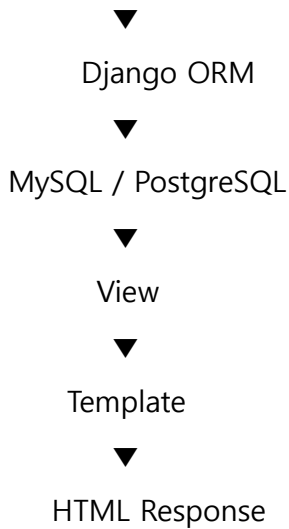


처럼 하나의 시스템에서 함께 사용할 수도 있습니다.

10. Django 를 이용한 실제 웹 서비스 구조

조금 더 실무적인 웹 애플리케이션은 다음과 같은 형태가 될 수 있습니다.

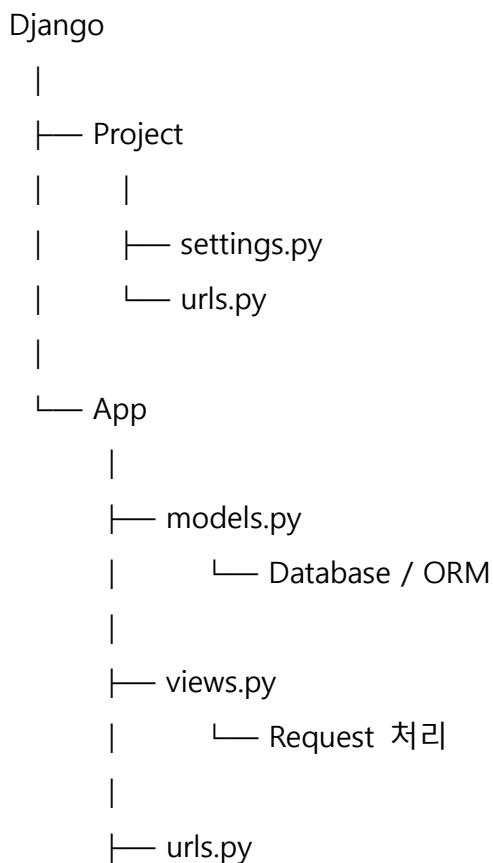




React 나 Vue 같은 프론트엔드 프레임워크를 사용한다면 Django 가 HTML 을 직접 만드는 대신 JSON 데이터를 제공하는 REST API 서버 역할을 담당하도록 설계할 수도 있습니다.

11. 핵심 정리

Django 를 처음 공부할 때는 모든 기능을 한꺼번에 외우기보다 다음 구조를 먼저 이해하는 것이 중요합니다.



```
|      └─ URL Mapping
|
└─ templates/
    └─ HTML 화면
```

핵심 개념을 한 문장으로 정리하면 **Django** 는 **Python** 을 이용하여 **URL 처리**, **비즈니스 로직**, **데이터베이스**, **HTML 화면**, **사용자 인증과 관리자 기능** 등을 체계적으로 개발할 수 있도록 지원하는 **종합 웹 프레임워크**입니다.